FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055
161_SIST. INTELIGENTES_N

P2

Relatório consolidado

Arquitetura e atividades do 2º bimestre

MangaCloud

Biblioteca pessoal e leitor web de mangás

github.com/Paulo794/Manga-Cloud
paulo-antunes-isw055.lapps.studio

Paulo Henrique Domingues Antunes

Avaliação individual · 2026.2

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Componentes e microsserviços 3
3.2 Onde roda 4
3.3 Fluxo de uma requisição 5
3.4 Fluxo de assinatura 6
3.5 Modelo de dados do domínio 7
3.6 Segurança 8
3.7 Observabilidade e CI/CD 8
3.8 Camadas IaaS, PaaS e SaaS do projeto 9
4 Quadro de entregas 10
5 Atividades realizadas 11
5.1 Atividade 7 — Plano Premium com Stripe 11
5.2 Atividade 8 — SaaS, PaaS, IaaS e documentação reversa 13
5.3 Atividade 9 — Fork e plano de negócio 15
5.4 Atividade 10 — Seu negócio no ar 17
5.5 Atividade 11 — SaaS completo com Stripe 22
6 Considerações finais 26
Declaração de autoria 26

1 Introdução

O segundo bimestre de Introdução à Computação em Nuvem reuniu as funcionalidades desenvolvidas no catálogo de filmes em um produto com domínio próprio. O MangaCloud resulta dessa evolução: autenticação, permissões, auditoria, armazenamento de arquivos e pagamentos passaram a apoiar uma biblioteca pessoal de mangás. O objetivo do produto é reunir a descoberta de obras, a organização em estantes privadas e a leitura disponível nas fontes configuradas, com sincronização de progresso para assinantes.

A necessidade atendida é a organização de leituras que normalmente ficam distribuídas entre abas, listas e serviços diferentes. Cada conta mantém suas próprias estantes, favoritos, capas e referências de leitura. A descoberta utiliza catálogos externos: o AniList fornece metadados e capas, enquanto o MangaDex fornece capítulos com páginas reais quando disponíveis. O catálogo Demo reúne conteúdo sintético identificado para demonstrar limites e assinatura sem apresentar uma simulação como conteúdo de uma obra real.

Este relatório consolida as atividades 7 a 11 e a arquitetura do produto. Apresenta componentes, fluxos, modelo de dados, segurança, camadas de nuvem e evidências verificáveis. O fechamento considera o código versionado, os registros históricos e as capturas disponíveis em 13/11/2026. A landing e rotas básicas foram verificadas no domínio público; os cenários autenticados e financeiros apresentados nas imagens foram executados em ambiente local de demonstração integrado ao Stripe em modo de teste.

> **Escopo das afirmações**
> A abertura pública do MangaCloud foi confirmada. A inspeção da stack no Portainer, o SHA efetivamente implantado e a repetição integral dos testes autenticados no domínio público não possuem comprovação anexada. Essas limitações permanecem explícitas nas fichas.

2 Metodologia

O desenvolvimento seguiu uma sequência incremental, preservando os serviços já existentes e substituindo o domínio de filmes por estantes e leitura. A inspeção para esta documentação considerou o código TypeScript/React, os arquivos Compose, as migrations SQL, os contratos OpenAPI e os relatórios de execução. O código-base consultado corresponde ao commit `fe1f700`. Os diagramas foram desenhados a partir desses arquivos; parâmetros previstos no Compose são diferenciados de informações observadas diretamente no ambiente público.

As datas planejadas foram transcritas do enunciado da P2. As datas realizadas utilizam o campo de registro do commit, quando a prova é um commit, e a criação da issue, quando a prova é a entrega pela issue. Os horários estão em **America/Sao_Paulo (UTC−03:00)**. A API do GitHub fornece datas em UTC; por exemplo, `2026-11-10T15:46:00Z` corresponde a 10/11/2026 às 12:46. Na atividade 7, a autoria do commit é 17:27:56 e seu registro é 17:33:42; este relatório adota o segundo horário, sem confundir os dois campos.

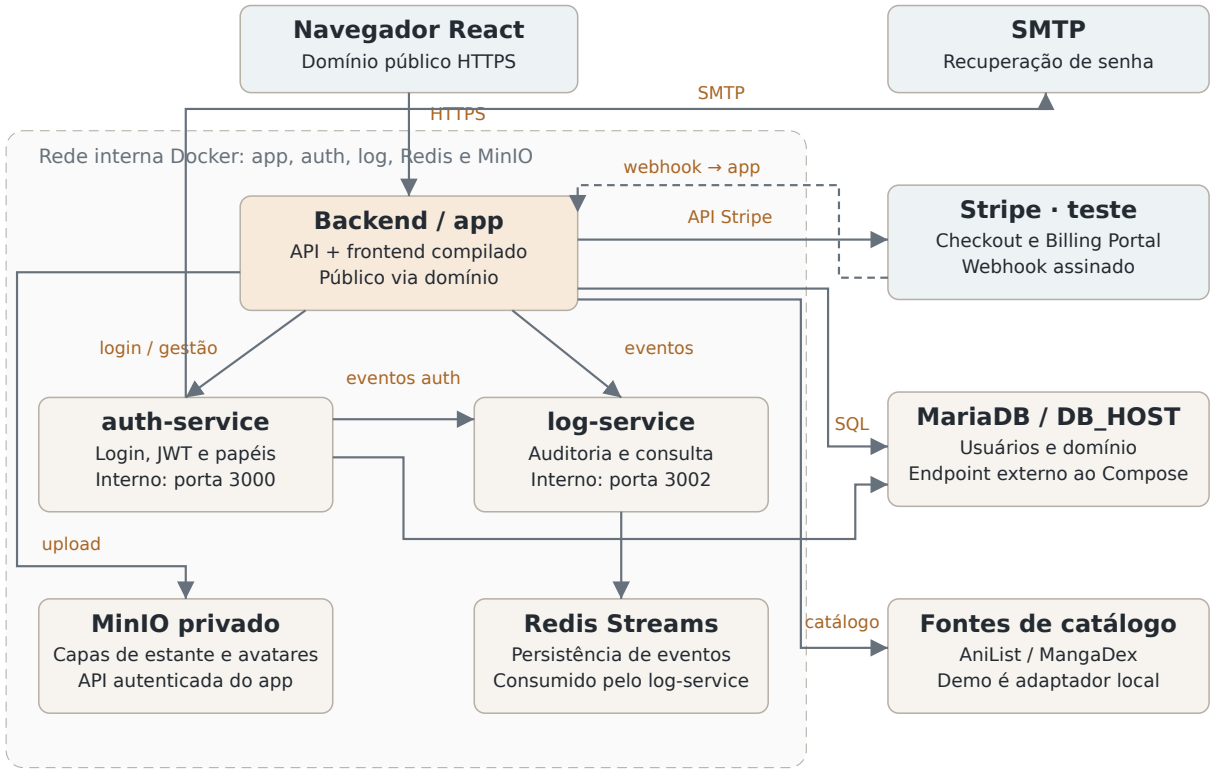
O histórico anterior foi preservado no repositório do produto. Por isso, as atividades 7 e 8 podem ser consultadas pelos mesmos hashes no repositório atual, embora tenham sido realizadas no catálogo anterior. Capturas históricas da atividade 7 foram recuperadas dos arquivos existentes naquele commit. Os prints das atividades 7 a 11 enviados pelo aluno mostram diretamente repositório, hash, autoria e horários no terminal. O aluno também forneceu capturas da documentação reversa e do README no GitHub; suas legendas identificam o conteúdo mostrado, sem atribuí-las automaticamente ao commit de entrega. As imagens de funcionamento foram selecionadas da pasta `fotos/` e copiadas sem modificar seu conteúdo.

A verificação pública, em 13/11/2026 às 22:48, foi somente de leitura: landing, saúde, prontidão, configuração de catálogo, Swagger e recusa de requisições sem sessão. Os resultados e URLs constam em `docs/p2/fontes-verificadas.json`. Os testes financeiros locais descritos nas fichas usam o Stripe real em modo de teste e Test Clocks; isso é diferente das respostas Stripe simuladas utilizadas nos testes de regressão offline. Datas de implementação, testes e entrega formal são registradas separadamente.

3 Arquitetura do SaaS

3.1 Componentes e microsserviços

O backend Express entrega a API e a interface React compilada pela mesma origem. O `auth-service` concentra cadastro, login, recuperação e gestão de papéis; o `log-service` recebe eventos internos e os persiste no Redis Streams. O MariaDB guarda o domínio, e o MinIO guarda arquivos enviados pelos usuários. Integrações externas são acessadas por adaptadores e pelo SDK do Stripe.



Setas representam chamadas. O navegador também abre o Checkout e o Portal hospedados pelo Stripe.

Figura 1: Componentes e chamadas do MangaCloud. A fronteira Docker foi extraída do Compose; o MariaDB publicado é acessado por um endpoint configurável.

Componente	Responsabilidade	Exposição prevista
App / backend	SPA React, API, regras do domínio, validação da sessão, planos e webhook.	Público pelo domínio.
auth-service	Credenciais, JWT, recuperação e alteração de papéis autorizada.	Rede interna Docker.
log-service + Redis	Recepção de eventos e consulta cronológica da auditoria.	Rede interna Docker.
MariaDB	Usuários, estantes, referências, progresso, assinaturas e idempotência.	Endpoint `DB_HOST`.
MinIO	Objetos privados: capas personalizadas e avatares.	App autenticado; console administrativo separado.
Stripe / fontes	Checkout, Portal e eventos; catálogo e leitura nas fontes.	Serviços externos.

O login atravessa o `auth-service`. Nas demais rotas autenticadas, o backend valida o JWT e reconsulta conta e papel no MariaDB. Assim, não existe uma chamada obrigatória ao serviço de login em cada operação de estante. O estado financeiro também é consultado no banco, independentemente do papel contido na interface.

3.2 Onde roda

O endereço público informado pelo aluno e verificado é paulo-antunes-isw055.lapps.studio. A resposta HTTP 200 abriu a landing do MangaCloud. O ambiente de implantação da disciplina utiliza a VM e a stack existente no Portainer; a inspeção administrativa dessa stack não integra as evidências disponíveis para este fechamento.

Aspecto	Estado comprovado ou configurado
Publicação HTTPS	Landing MangaCloud acessível no subdomínio informado.
Código-fonte	Repositório público Manga-Cloud; referência de análise: commit `fe1f700`. O commit implantado não é exposto pelo serviço.
Compose de implantação	Serviços `catalogo`, `auth-service`, `log-service`, `redis` e `minio`; MariaDB acessado por configuração externa.
Identificadores herdados	Aliases físicos `catalogo_tom_hanks`, `auth_tom_hanks`, `log_tom_hanks`, `redis_tom_hanks`, `minio_tom_hanks` e rede `tom_hanks_net` preservam o contrato da instalação anterior.
Memória	O Compose declara limite de 1 GB por serviço. A aplicação efetiva desses limites no Portainer ainda precisa de captura.
Ambiente local	`compose.local.yml` utiliza banco, volumes e rede próprios, com acesso em loopback e etapa `init` para migration/seed.

Os nomes físicos herdados não representam funcionalidades de filmes em uso. O frontend e as rotas do produto passaram ao domínio MangaCloud. A manutenção dos aliases deve ser tratada na operação da mesma stack, preservando volumes e conectividade.

As variáveis abaixo são apresentadas **somente por nome**, sem valores ou credenciais:

Grupo	Variáveis principais
Aplicação e sessão	`APP_URL`, `PORT`, `JWT_SECRET`, `CATALOG_PROVIDER`, `AUTH_SERVICE_URL`, `LOG_SERVICE_URL`, `INTERNAL_SERVICE_TOKEN`.
Banco e auditoria	`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `REDIS_URL`, `REDIS_PASSWORD`, `AUDIT_STREAM_MAXLEN`.
Objetos e email	`MINIO_ENDPOINT`, `MINIO_PORT`, `MINIO_REGION`, `MINIO_ROOT_USER`, `MINIO_ROOT_PASSWORD`, `MAILTRAP_HOST`, `MAILTRAP_PORT`, `MAILTRAP_USER`, `MAILTRAP_PASS`, `MAIL_FROM`.
Stripe teste	`STRIPE_DEMO_ENABLED`, `STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID`, `STRIPE_WEBHOOK_SECRET`.

Na consulta pública, `/health`, `/ready` e `/api-docs/` responderam 200. `/api/estantes` e `/api/admin/metricas` responderam 401 sem autenticação. Essas respostas comprovam esses endpoints; não demonstram sozinhas o estado de todos os containers ou o fluxo completo de assinatura em produção.

3.3 Fluxo de uma requisição

A criação de uma estante exemplifica a relação entre identidade, regra comercial e persistência. O navegador envia `POST /api/estantes` com nome e descrição. O backend verifica a assinatura JWT com HS256 e consulta o usuário atual, garantindo que a conta exista e que o papel continue válido. O dono do registro vem da sessão, sem aceitar um `usuario_id` escolhido pelo cliente.



Figura 2: Percurso de criação de uma estante após o login. O limite é aplicado na API, inclusive em chamadas diretas.

O backend inicia uma transação e bloqueia a linha do usuário para serializar criações concorrentes. Consulta a assinatura vigente e, no Free, conta as estantes existentes. Até três estantes são permitidas; uma criação adicional recebe `403 PLAN_REQUIRED`, com mensagem sobre o limite e o Pro. O bloqueio impede que duas requisições simultâneas ultrapassem a cota pela mesma contagem inicial.

Quando a operação é aceita, a estante é inserida com o dono autenticado e a transação é confirmada. A resposta retorna HTTP 201 e o ID criado. O evento `estante.criada` é enviado ao `log-service`, com autor, recurso e resultado, e passa a ser consultado no painel administrativo pelo Redis Streams.

O transporte de auditoria é assíncrono e possui timeout: uma falha no log não desfaz uma estante já salva. Essa escolha mantém o uso da biblioteca disponível, mas o envio de eventos é de melhor esforço, sem uma outbox transacional que garanta recuperação de toda falha de transporte. A documentação não atribui ao log uma garantia que o código não implementa.

Ao abrir `GET /api/estantes/:id`, o filtro combina o ID solicitado com o usuário autenticado. Se o registro pertencer a outra conta, a API devolve 404. Nas rotas administrativas, a ausência do papel necessário produz 403. São verificações diferentes: propriedade protege a biblioteca pessoal, e RBAC protege funções de administração.

3.4 Fluxo de assinatura

O Free permite até três estantes pessoais, as três primeiras posições canônicas das obras Demo e a consulta da biblioteca. O Pro de teste custa R$ 9,90/mês e libera estantes ilimitadas, todos os capítulos Demo e gravação automática da página. O papel administrativo não concede Pro. O acesso externo permanece separado: AniList oferece prévias ilustrativas, e MangaDex oferece páginas reais disponíveis sem paywall comercial do produto.

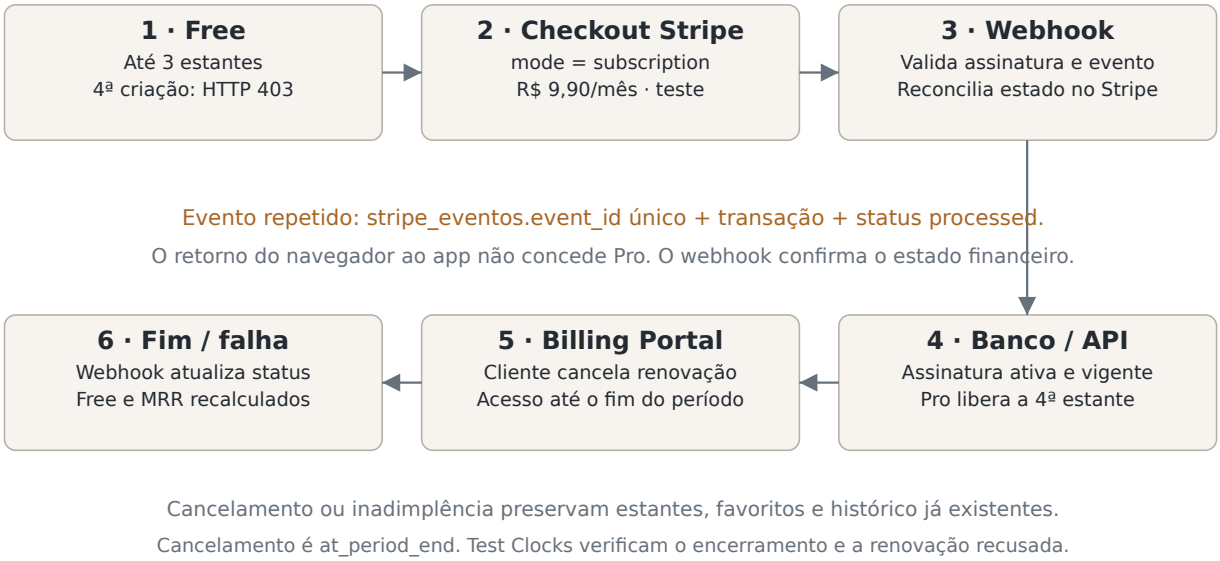


Figura 3: Ciclo de assinatura recorrente em modo de teste, com concessão no servidor e cancelamento pelo Portal.

A rota `POST /api/stripe/checkout` consulta o preço no Stripe, confere BRL, recorrência mensal e valor esperado, e cria uma sessão com `mode=subscription`. O cliente Stripe é associado ao usuário por `stripe_customer_id` e por `billing_identity`, uma identidade durável que evita confusão entre bancos que reutilizem IDs. Sessões abertas da mesma conta são reaproveitadas quando compatíveis, reduzindo checkouts duplicados.

O Stripe hospeda o formulário de cartão. O servidor recebe IDs de cliente, sessão e assinatura, sem receber o número do cartão. O retorno para `/sucesso` apenas solicita a atualização do estado no backend; não altera o plano por parâmetros de URL. A liberação depende do webhook processado e da assinatura vigente no MariaDB.

Evento obrigatório	Tratamento implementado
`checkout.session.completed`	Reconcilia a assinatura e vincula a cobrança à conta; pagamento e vigência determinam Pro.
`customer.subscription.updated`	Atualiza situação e cancelamento agendado a partir do estado atual no Stripe.
`customer.subscription.deleted`	Registra o término e remove o acesso Pro; mantém os dados pessoais existentes.
`invoice.payment_failed`	Registra falha/inadimplência, revoga Pro quando não há pagamento vigente e exibe orientação ao usuário.

O receptor usa o corpo bruto da requisição para validar `Stripe-Signature`, rejeita eventos live e verifica a propriedade da assinatura. O `event_id` é chave primária de `stripe_eventos`; processamento e atualização financeira ocorrem em transação. Uma repetição de evento já processado não reaplica o efeito. Falhas retornam erro para permitir reenvio, e o estado remoto é reconciliado após a serialização por usuário para reduzir efeitos de eventos fora de ordem.

O acesso Pro é calculado com assinatura `active` e `acesso_ate` posterior ao instante atual. O campo legado `usuarios.is_premium` é uma projeção de compatibilidade e não autoriza recursos. O webhook também considera fatura paga e preço configurado antes de conceder a vigência.

`POST /api/stripe/portal` abre o Billing Portal do cliente autenticado após conferir sua propriedade. A política testada é cancelamento ao fim do período: o acesso permanece até a vigência, e o evento terminal devolve a conta ao Free. O código trata tanto `cancel_at_period_end` quanto `cancel_at` ao reativar. Assinaturas vencidas ou inadimplentes deixam de contar no MRR, calculado pelo número de assinaturas Pro vigentes multiplicado por 990 centavos.

3.5 Modelo de dados do domínio

O modelo relacional mantém os dados privados e as referências externas. A migration `002_dominio_mangacloud.sql` cria o domínio e a `004_identidade_cobranca.sql` acrescenta a identidade durável de cobrança. O runner registra versões e checksums; o seed é idempotente e utiliza referências estáveis para preservar registros já editados.

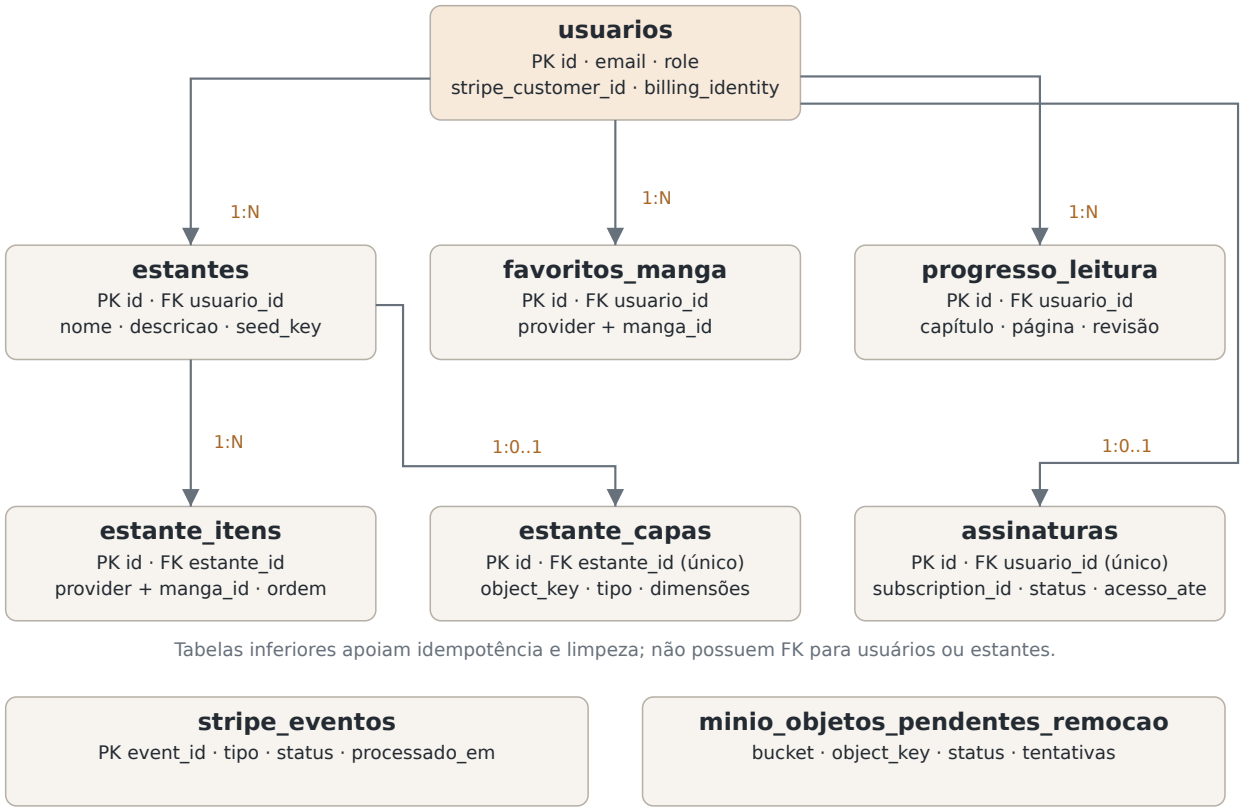


Figura 4: Relacionamentos centrais do domínio. Referências a obras externas usam o par provedor e identificador, sem FK para uma tabela local de catálogo.

Tabela / vínculo	Finalidade e integridade
`usuarios`	Identidade, credenciais com hash, papel e referência do cliente Stripe; `billing_identity` é única.
`estantes.usuario_id`	FK para `usuarios.id`; delimita o dono de cada biblioteca.
`estante_itens.estante_id`	FK para estante; par `provider + manga_id` único por estante; ordem dos itens.
`estante_capas.estante_id`	FK única para estante; guarda chave do objeto, MIME, dimensões, tamanho e nome original.
`favoritos_manga`	FK para usuário e unicidade de referência por conta.
`progresso_leitura`	FK para usuário; obra, capítulo, idioma, página e revisão para atualização concorrente.
`assinaturas.usuario_id`	FK única para usuário, com IDs Stripe, preço, status, vigência e cancelamento.
`stripe_eventos`	Chave `event_id` e estado de processamento para idempotência.

O catálogo não é copiado para o MariaDB: estantes e favoritos armazenam referências qualificadas pelo provedor. Os metadados são hidratados na consulta; uma indisponibilidade externa mantém a referência e a possibilidade de editar a estante. Páginas de capítulos não são armazenadas no MinIO. O proxy das capas MangaDex mantém apenas cache limitado em memória.

O MinIO guarda capas personalizadas e avatares em buckets privados. As alterações de arquivos são coordenadas com o SQL e com a fila `minio_objetos_pendentes_remocao`: substituições e exclusões registram os objetos antigos para limpeza. A atualização de progresso usa revisão esperada; duas gravações incompatíveis não se sobrepõem silenciosamente, pois a versão desatualizada recebe 409.

3.6 Segurança

As permissões são aplicadas no backend e no serviço de autenticação. O cadastro público cria leitor; valores de papel ou plano enviados no formulário não promovem a conta. Após validar o JWT, a API reconsulta o papel no banco, de modo que uma alteração administrativa afeta chamadas seguintes mesmo com um token emitido anteriormente.

Ação	Leitor	Curador	Admin
CRUD da própria estante e capa	Sim	Sim	Sim
Favoritos e leitura próprios	Sim	Sim	Sim
Diagnóstico das fontes	Não	Sim	Sim
Alterar papéis e consultar auditoria	Não	Não	Sim
Abrir estante privada de terceiro	Não	Não	Não
Benefício Pro por possuir o papel	Não	Não	Não

O isolamento utiliza `usuario_id` obtido da sessão nas consultas, e itens/capas são acessados por vínculo com uma estante do mesmo dono. O 404 evita revelar a biblioteca de outro usuário. Rotas administrativas exigem papel permitido e devolvem 403; chamadas sem sessão recebem 401. O limite do Free é aplicado mesmo quando a requisição ignora a interface.

Uploads aceitam JPEG, PNG e WebP, com limite de 5 MiB, até 4096 pixels por dimensão e 16 milhões de pixels. O backend confere estrutura e dimensões, decodifica e reencoda a imagem. A propriedade da estante é validada antes de receber seu upload. Objetos são entregues por API autenticada, sem expor credenciais do MinIO ao navegador.

Segredos ficam em variáveis do ambiente de execução; `.env` e os arquivos de configuração local não devem integrar o repositório. A comunicação com o `log-service` usa token interno, e o webhook financeiro exige assinatura válida. As capturas locais utilizam contas sintéticas e omitem tokens. Não há número de cartão armazenado pelo app.

Há limitações de segurança e operação a reconhecer. O logout remove a sessão do navegador e registra o evento, mas não existe lista de revogação imediata de JWT. A proteção permanece vinculada à expiração e à revalidação de conta/papel. A configuração de email e de URLs de retorno no ambiente público precisa ser conferida para que links de recuperação e cobrança usem o domínio correto.

3.7 Observabilidade e CI/CD

O projeto possui Swagger/OpenAPI para a API e o serviço de autenticação. `/health` verifica disponibilidade do processo; `/ready` consulta o banco e a estrutura de estantes. O Compose define healthchecks para aplicação e serviços de apoio. A consulta pública confirmou essas duas rotas do app com HTTP 200, sem afirmar por isso que todos os healthchecks da VM foram inspecionados.

Os eventos de negócio incluem criação/edição/exclusão de estantes, capas, favoritos, leituras liberadas ou bloqueadas, limite Free, progresso, atualização de assinatura e alteração de papel. O Redis Streams associa os registros ao instante de inserção. O painel admin apresenta horário, ator, ação, recurso, detalhe e resultado; as métricas SaaS mostram usuários Free/Pro, assinaturas ativas e MRR de teste.

Os relatórios existentes registram builds, lint e testes de integração com MariaDB, MinIO e Redis isolados, além de navegador e Stripe teste. Testes offline simulam explicitamente respostas do Stripe; a rodada ao vivo de 13/11 utiliza Checkout, Portal, webhooks reais e Test Clocks. Não foi identificada uma pipeline GitHub Actions neste repositório: os executores locais não são apresentados como CI/CD automatizado.

3.8 Camadas IaaS, PaaS e SaaS do projeto

A classificação depende da fronteira de responsabilidade. A VM disponibilizada no contexto da disciplina fornece infraestrutura; o ambiente de containers oferece os runtimes e serviços consumidos pelo código; o MangaCloud entrega a experiência final ao leitor. O uso de uma imagem pronta não transfere automaticamente a operação do serviço para um provedor de PaaS.

Camada	Aplicação no projeto	Responsabilidade
IaaS	VM, CPU, memória, disco e rede da infraestrutura acadêmica.	A infraestrutura fornece a máquina; SO, Docker, regras e implantação precisam de administração.
PaaS	Plataforma de execução composta por Node.js, rede Docker, MariaDB, Redis e MinIO consumidos pelo código.	Plataforma autogerenciada no contexto do projeto. Não há comprovação de contratação de PaaS ou DBaaS gerenciado.
SaaS próprio	MangaCloud: conta, biblioteca privada, leitura e recursos Free/Pro.	O usuário consome a aplicação; a operação do produto permanece sob responsabilidade do projeto.
Serviços externos	Stripe, SMTP e APIs de catálogo.	Os fornecedores operam seus serviços. O projeto administra integração, segredos e regras que aplica aos dados recebidos.

Stripe Checkout e Billing Portal são serviços prontos utilizados pelo app. AniList e MangaDex são dependências externas de catálogo e conteúdo, com disponibilidade independente da biblioteca privada. Se uma fonte falhar, as referências pessoais continuam no banco; isso diferencia a responsabilidade sobre os dados do usuário da responsabilidade sobre um acervo externo.

O Portainer facilita a administração da stack, mas não comprova sozinho escalabilidade, alta disponibilidade ou capacidade sob carga. A solução documentada é uma implantação acadêmica com limites declarados no Compose, somada a testes locais isolados. Não há evidência de cluster, autoscaling ou arquitetura multirregião, e essas capacidades não são atribuídas ao produto.

4 Quadro de entregas

O quadro é gerado a partir dos mesmos dados utilizados nas fichas da seção 5. O fuso é UTC−03:00. Nas atividades 10 e 11, a data marca o **marco de implementação versionado**, e a situação parcial registra que a entrega formal e parte das evidências operacionais não foram comprovadas.

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium com Stripe	09/10/2026	09/10/2026 17:33:42	Entregue
8	SaaS, PaaS, IaaS e documentação reversa	16/10/2026	23/10/2026 15:08:48	Entregue com atraso
9	Fork e plano de negócio	23/10/2026	10/11/2026 12:46:00	Entregue com atraso
10	Seu negócio no ar	06/11/2026	13/11/2026 22:25:28[1]	Parcial; prazo ultrapassado
11	SaaS completo com Stripe	13/11/2026	13/11/2026 22:24:49[1]	Parcial; entrega final pendente

Na atividade 8, a documentação foi criada em 23/10 e ligada ao README no mesmo dia, depois do prazo de 16/10. A atividade 9 tem a issue aberta em 10/11, depois do prazo de 23/10; o plano de negócio final do MangaCloud foi atualizado em 11/11. Essas datas são preservadas sem substituir o prazo planejado pelo dia da implementação.

Nas atividades 10 e 11, a publicação de commits e as demonstrações locais comprovam trabalho realizado. A verificação pública confirmou o produto acessível, mas não cobre os testes autenticados completos, o limite efetivo de memória no Portainer e o Dashboard Stripe atualizado. A consulta às issues públicas encontrou a entrega da atividade 9; não encontrou issues finais das atividades 10 e 11 naquele momento. Por isso, nenhuma delas é declarada integralmente entregue neste fechamento.

> **Critério de situação**
> “Parcial” identifica implementação existente com entrega final pendente. O relatório mantém as pendências e os atrasos visíveis, conforme o requisito de honestidade do enunciado. A avaliação e os descontos são definidos pelo professor.

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium com Stripe

Descrição	Integração inicial de Checkout e webhook, com identificação Premium no perfil.
Data planejada	09/10/2026
Data realizada	09/10/2026 17:33:42
Situação	Entregue
Evidência	Commit histórico consultável e imagens incluídas na versão original.
Links diretos	commit `1b11f42` Resultado histórico: perfil Premium

O que foi feito. A integração do catálogo anterior passou a criar uma sessão de Checkout hospedada pelo Stripe e a validar o webhook com corpo bruto. O evento `checkout.session.completed` vinculava a conta e marcava `is_premium`, exibindo o selo no perfil. Embora a sessão já usasse `mode=subscription`, o estado da aplicação ainda se concentrava nesse booleano, sem o controle completo de vigência e inadimplência exigido depois na atividade 11.

```
Repositório: Paulo794/Manga-Cloud
Commit: 1b11f421d01aefe0252ec40a776a24288ff92ca0
Autor: Paulo Henrique
Data de autoria: 09/10/2026 17:27:56 -0300
Data do commit: 09/10/2026 17:33:42 -0300
Descrição: okk
```

Figura 5: Atividade 7: print do terminal com repositório, hash e horários do commit 1b11f42.

Resultado. As imagens abaixo pertencem ao commit de 09/10, recuperadas do histórico. Elas demonstram o Checkout e o perfil Premium do catálogo anterior; não são apresentadas como telas atuais do MangaCloud.

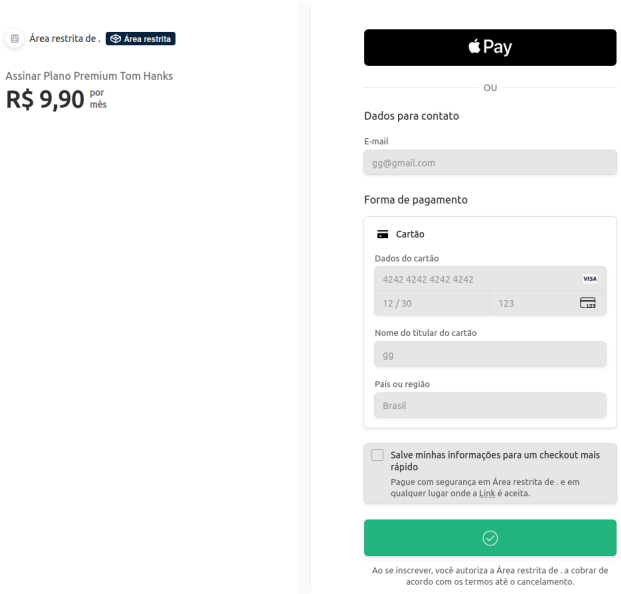


Figura 6: Atividade 7: captura histórica do Checkout Stripe com cartão de teste, preservada no commit original.

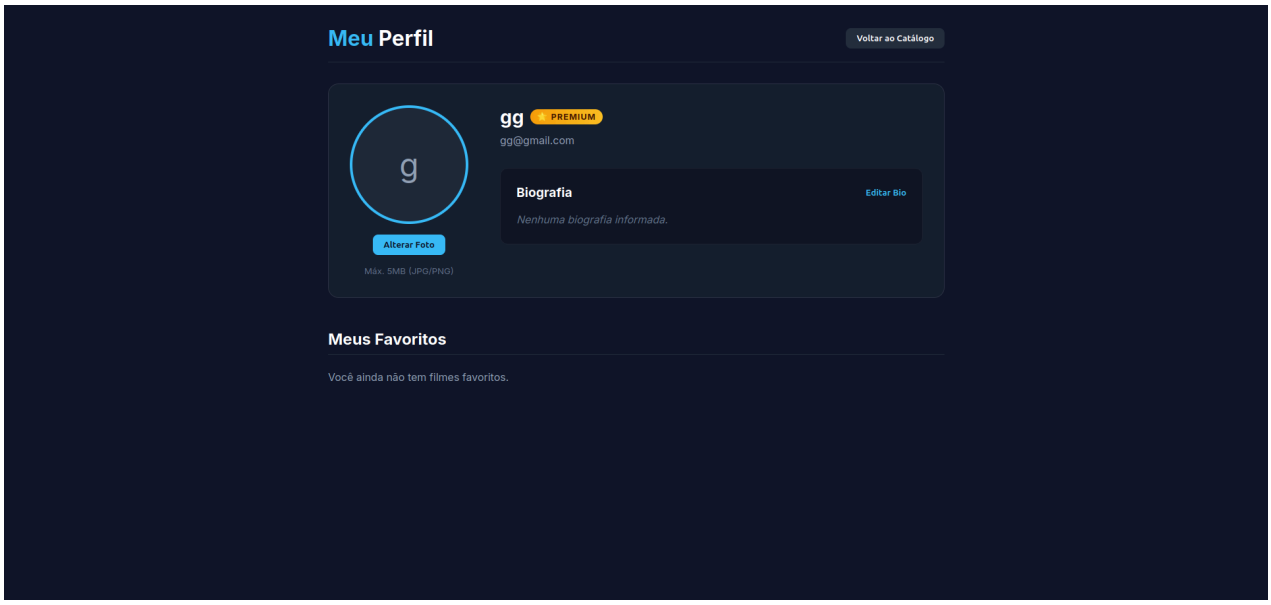


Figura 7: Atividade 7: perfil exibindo o selo Premium após o fluxo registrado.

Dificuldades e solução técnica. A assinatura do webhook exige o corpo bruto recebido, enquanto o parser JSON padrão do Express transforma o conteúdo. A rota foi registrada antes do parser. O vínculo com o usuário utilizou os metadados enviados ao Checkout, evitando depender somente do retorno do navegador.

Limite da evidência. As capturas registram a implementação histórica. O ciclo completo de renovação, cancelamento, idempotência e planos pertence à evolução posterior. Não foi localizada uma issue específica da atividade 7; a prova utilizada é o commit e seus arquivos versionados.

5.2 Atividade 8 — SaaS, PaaS, IaaS e documentação reversa

Descrição	Descrição funcional, arquitetura, regras extraídas do código e modelo de dados do sistema anterior.
Data planejada	16/10/2026
Data realizada	23/10/2026 15:08:48
Situação	Entregue com atraso
Evidência	Commit que liga DOCUMENTACAO.md no README; conteúdo histórico consultável.
Links diretos	commit `eb76427` DOCUMENTACAO.md na versão da entrega

O que foi feito. O documento técnico foi criado no commit `a4f9986` em 23/10/2026 às 15:06:26. O commit seguinte acrescentou o link ao README e a classificação de camadas; por concluir esse requisito, seu horário é a data realizada da ficha. O texto descreveu funcionalidades por perfil, comunicação entre serviços, regras de negócio e armazenamento em MariaDB, Redis e MinIO.

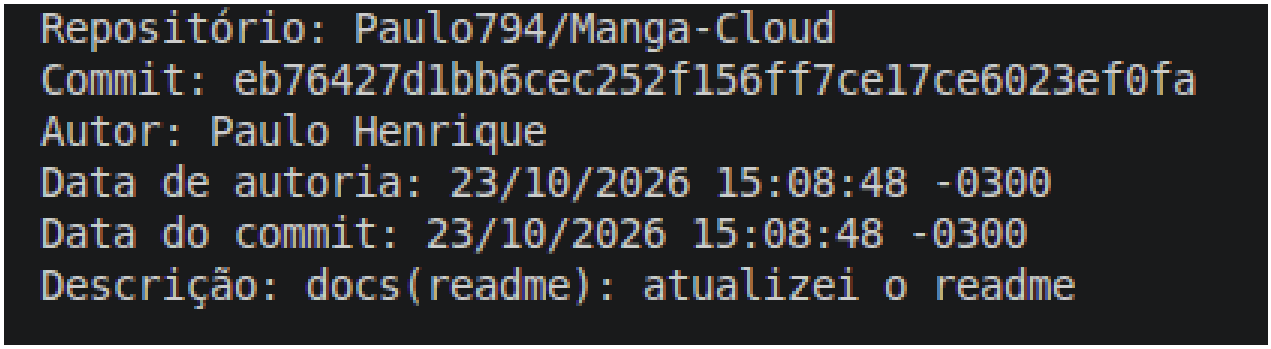


Figura 8: Atividade 8: print do terminal com repositório, hash e horários do commit eb76427.



Figura 9: Atividade 8: documentação reversa do catálogo de filmes aberta no GitHub. Recorte da captura fornecida pelo aluno.

Dificuldades e solução técnica. A documentação reversa exigiu diferenciar o comportamento do código das premissas do ambiente. O material histórico adotou marcadores de origem da informação. Na P2, essa distinção é mantida ao separar parâmetros do Compose, resultados locais e observações do domínio público.

Situação e atualização. A entrega ocorreu após o prazo planejado. O `DOCUMENTACAO.md` atual foi atualizado para o MangaCloud; a ficha aponta a versão histórica do commit para que a evolução do conteúdo não apague a evidência da atividade 8. A issue "atividade 8" no repositório anterior complementa o registro, mas a data adotada aqui vem do commit.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório do produto com histórico preservado, proposta de valor, planos, Canvas e backlog.
Data planejada	23/10/2026
Data realizada	10/11/2026 12:46:00
Situação	Entregue com atraso
Evidência	Issue de entrega nº 7 e commit do plano final MangaCloud.
Links diretos	Issue: Entrega — Fork e plano de negócio Plano de negócio no commit de95a30

O que foi feito. O repositório Manga-Cloud preservou o histórico do projeto anterior e passou a organizar a proposta de biblioteca e leitura. O README detalha o público, o problema, alternativas, solução, modelo de receita e métricas; o backlog possui issues para dados, autorização, catálogo, interface, arquivos, auditoria e assinatura. A issue de entrega menciona o professor e contém capturas do histórico e das issues.

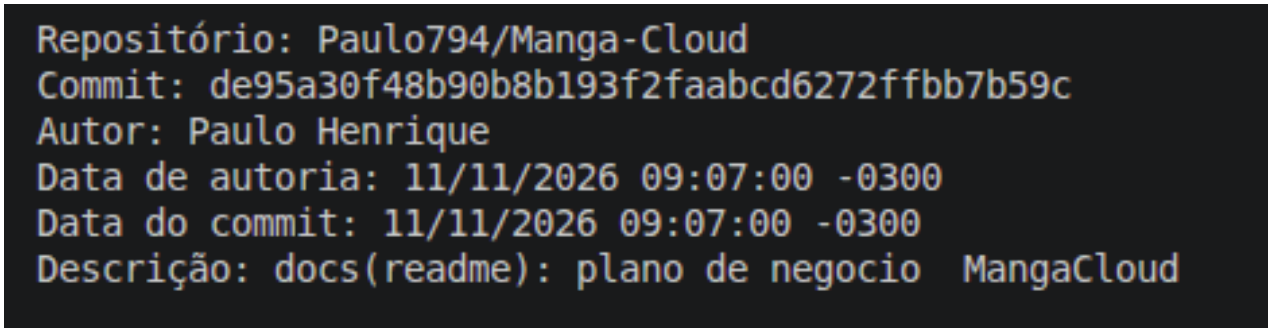


Figura 10: Print enviado pelo aluno: plano de negócio MangaCloud versionado em 11/11/2026 às 09:07:00, hash de95a30.

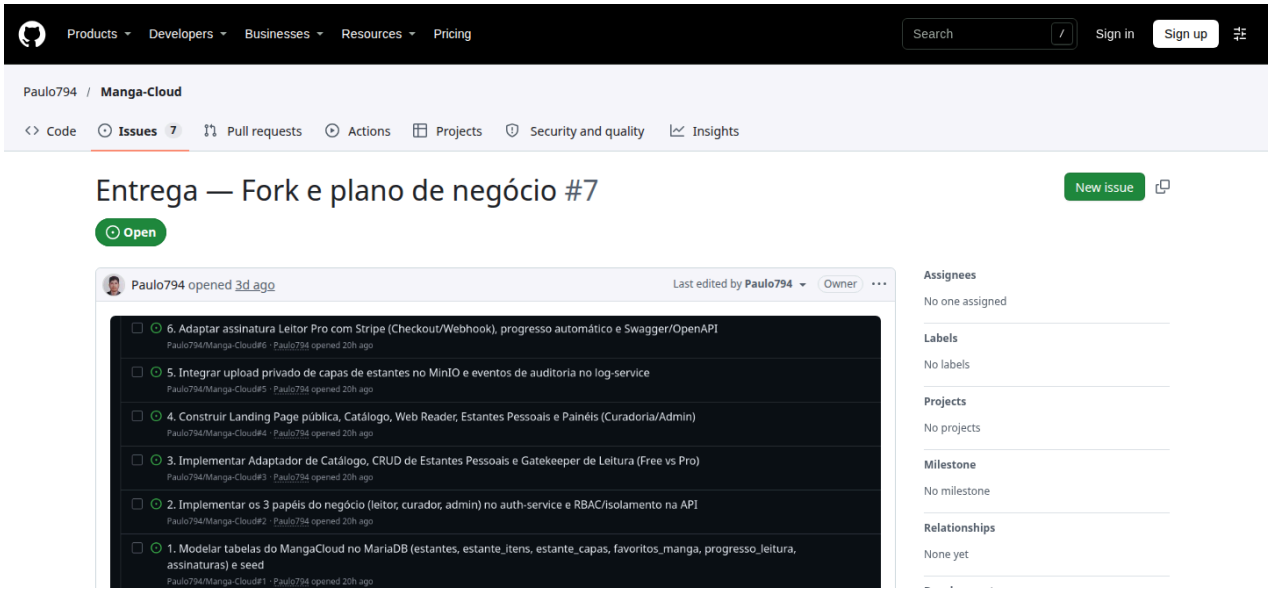


Figura 11: Atividade 9: issue pública de entrega com o histórico e o backlog apresentados.

A data da ficha corresponde à abertura da issue em 10/11. O commit de 11/11 é uma atualização posterior do plano final, e não substitui a data de criação da issue. O repositório foi criado em 10/11, preservando os commits de outubro do projeto anterior.



Figura 12: Atividade 9: README do MangaCloud com a proposta do produto e links para as atividades 10 e 11, na versão dcbda73 exibida no GitHub. Recorte da captura fornecida pelo aluno.

Dificuldades e solução técnica. A mudança de domínio exigiu definir um recurso próprio, além de consultar um catálogo externo. As estantes pessoais passaram a ser o CRUD principal, com itens, capas privadas e isolamento por conta. O plano separou os papéis do negócio dos benefícios de assinatura, preservando os serviços anteriores.

5.4 Atividade 10 — Seu negócio no ar

Descrição	Aplicação do domínio MangaCloud: estantes, RBAC, isolamento, arquivos, auditoria e landing pública.
Data planejada	06/11/2026
Data realizada	13/11/2026 22:25:28[1]
Situação	Parcial; prazo ultrapassado
Evidência	Commit e resultados locais; landing e rotas básicas verificadas no domínio público.
Links diretos	commit `dcbda73` Sistema público Matriz de evidências da atividade 10

O que foi feito. O catálogo anterior foi substituído por um produto com landing, catálogos AniList/MangaDex/Demo, obra, leitor, CRUD de estantes e capas privadas. O banco contém tabelas relacionadas do domínio, e as ações passam a gerar eventos de negócio. Os papéis leitor, curador e admin são independentes do plano Free/Pro. O servidor aplica 403 em funções sem permissão e 404 ao acessar uma biblioteca alheia.

```
Repositório: Paulo794/Manga-Cloud
Commit: dcbda73c4d5de4b941d90ba12ebcf071a6ad1db6
Autor: Paulo Henrique
Data de autoria: 13/11/2026 22:25:28 -0300
Data do commit: 13/11/2026 22:25:28 -0300
Descrição: docs: atualiza README e organiza evidencias das atividades 10 e 11
```

Figura 13: Print enviado pelo aluno: documentação e evidências das atividades 10 e 11 versionadas em 13/11/2026 às 22:25:28.



Figura 14: Landing pública observada em paulo-antunes-isw055.lapps.studio, em 13/11/2026, sem autenticação. Recorte.

Data e situação. O indicador [1] identifica o horário do marco versionado, sem convertê-lo em entrega final da atividade. A abertura pública foi confirmada na preparação da P2; a evidência disponível não identifica quando ocorreu o redeploy nem o SHA implantado.

Recurso principal e armazenamento. As capturas abaixo vieram de `fotos/` e mostram uma instância local isolada. Demonstram listagem, itens e capa privada vinculada à estante; a interface recebe a imagem pela API autenticada do app.

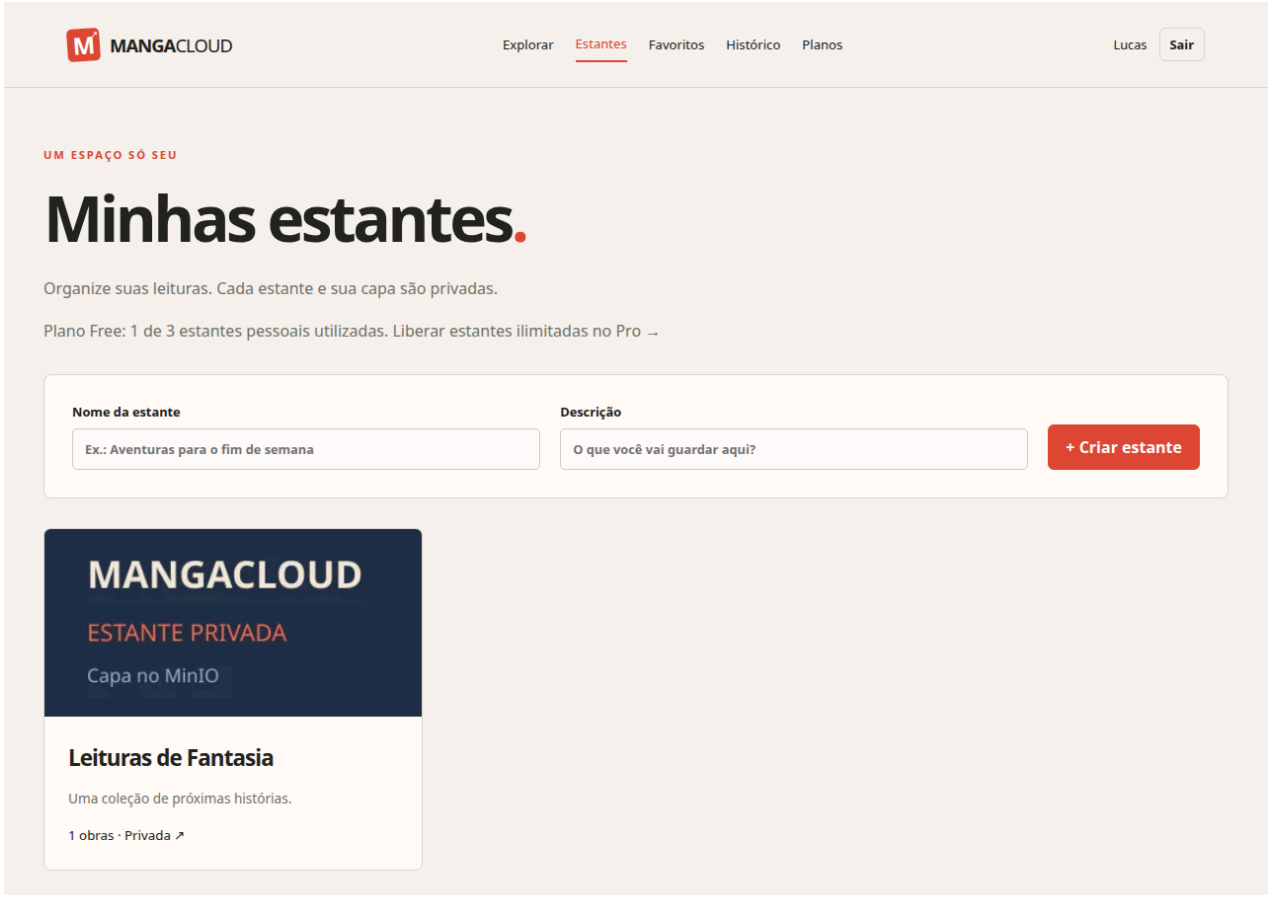


Figura 15: Atividade 10: estantes pessoais e capa do MinIO em ambiente local, 13/11/2026.

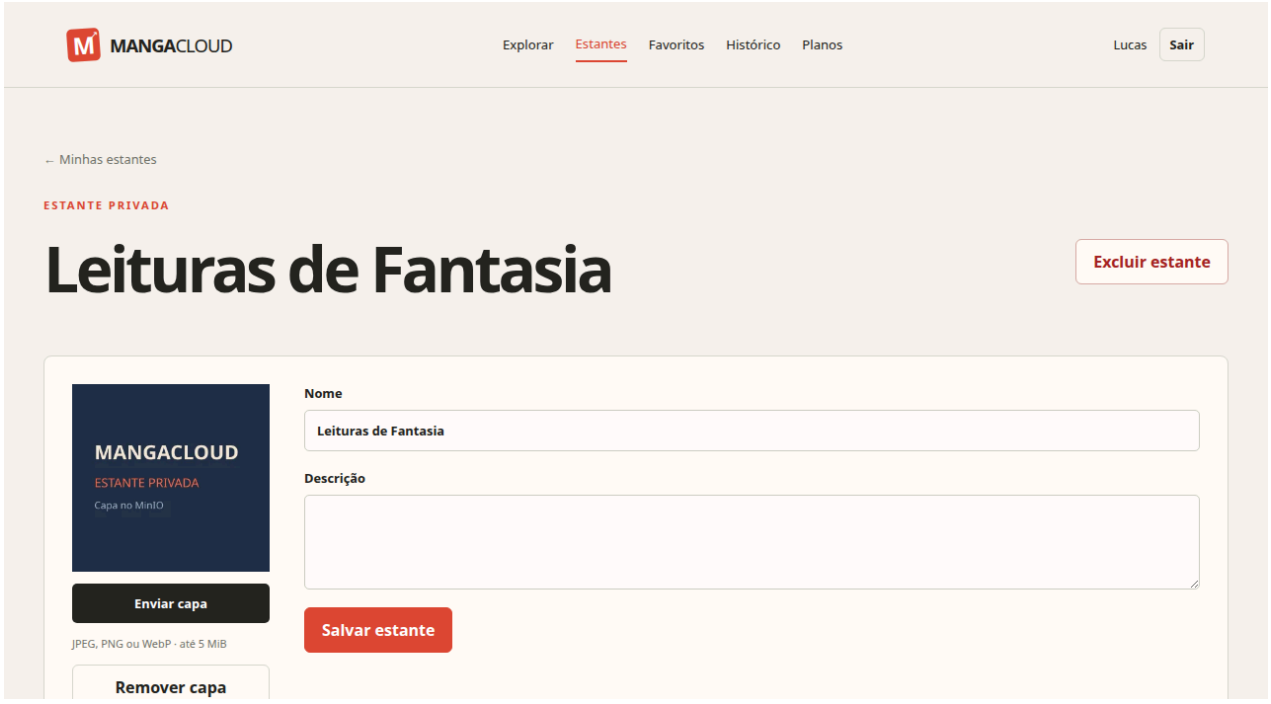


Figura 16: Detalhe e edição da estante com capa privada e obra adicionada. Recorte superior da captura local.

Permissões e isolamento. As chamadas diretas pelo Swagger exibem respostas reais da API local. O registro de data HTTP usa GMT/UTC, enquanto as fichas usam UTC−03. Os tokens foram omitidos na captura de origem.

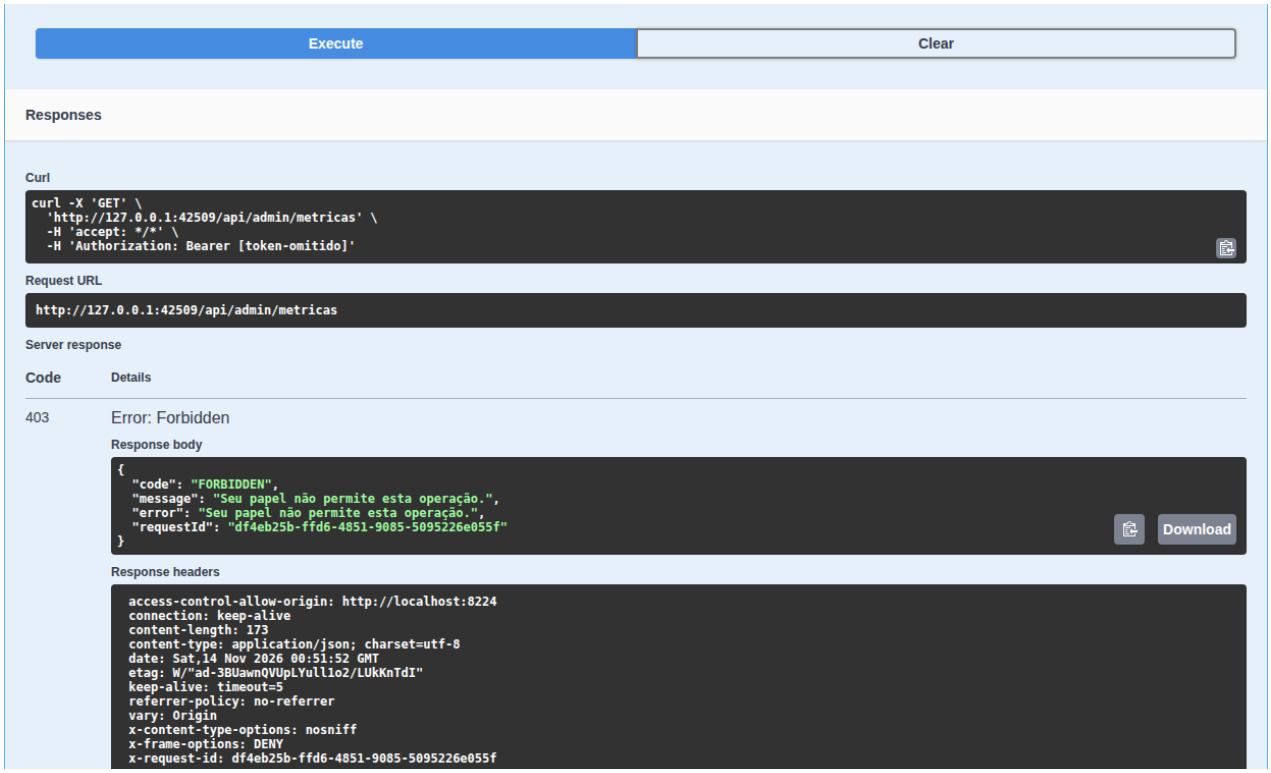


Figura 17: Leitor tenta GET /api/admin/metricas e recebe 403 FORBIDDEN. Recorte da requisição e resposta local.

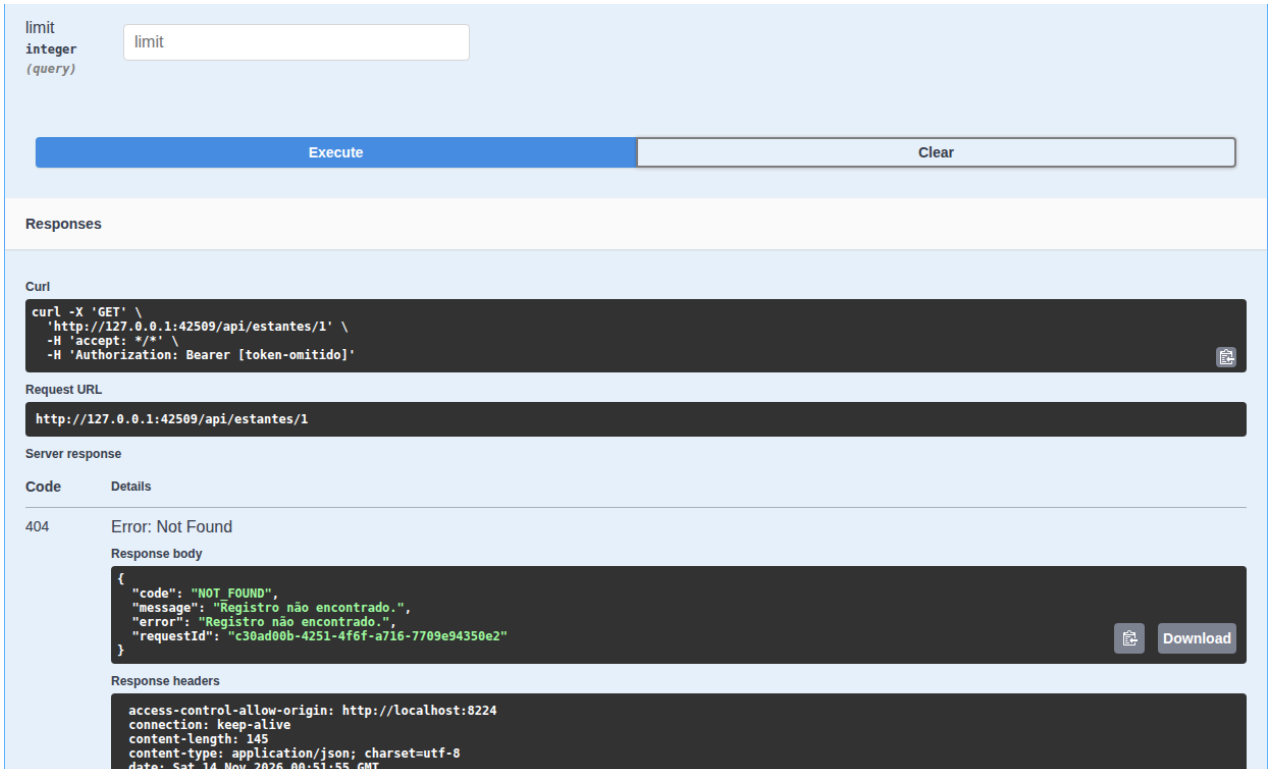


Figura 18: Conta B tenta abrir a estante da Conta A e recebe 404 NOT_FOUND. Recorte da requisição e resposta local.

Catálogo e leitura. As fontes são identificadas em todas as referências. O AniList oferece catálogo, capas e uma prévia ilustrativa; a captura MangaDex mostra uma página real disponível em português. O Demo permanece separado para demonstrar o bloqueio de plano.

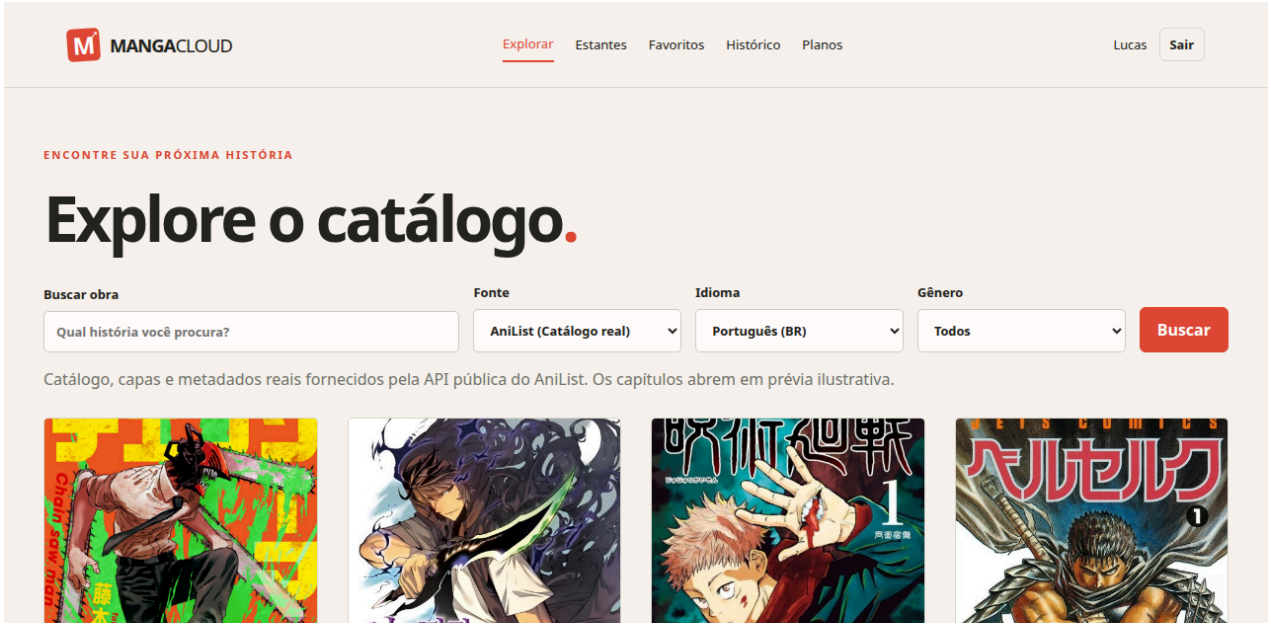


Figura 19: Catálogo AniList com metadados e capas reais. Recorte superior da captura local.

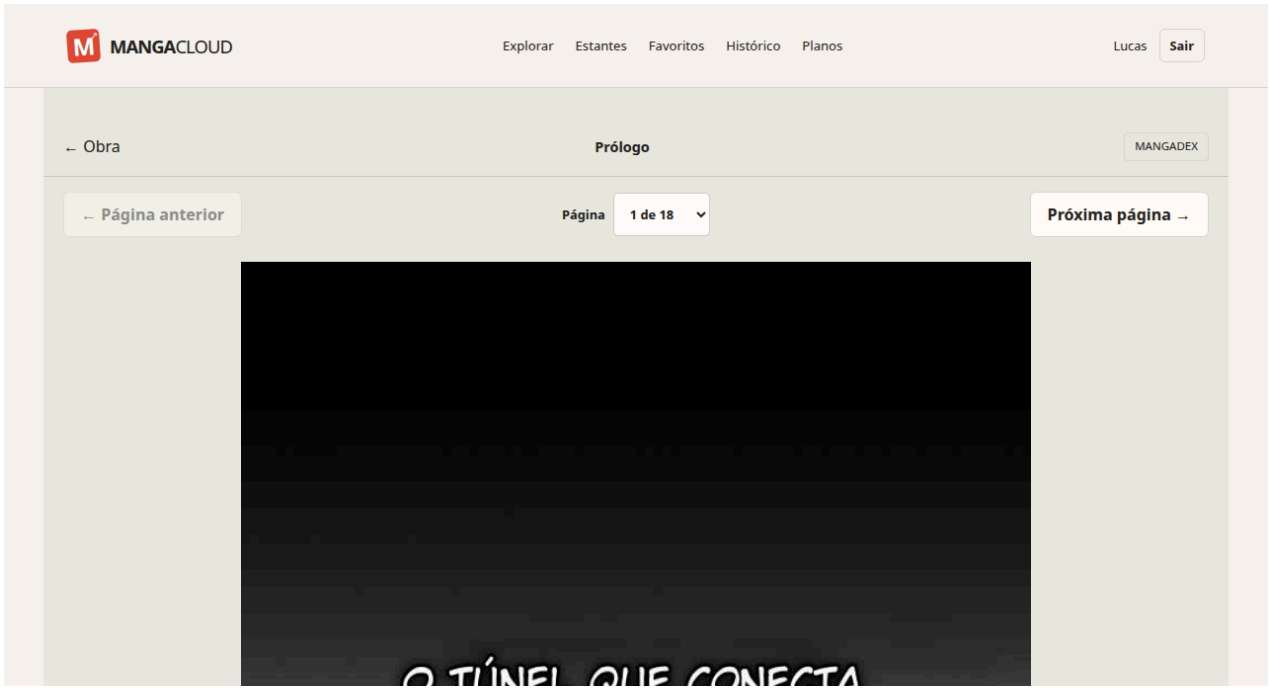


Figura 20: Leitor MangaDex exibindo página real de capítulo, em ambiente local. Recorte da captura.

Dificuldades e solução técnica. O acesso às fontes apresentou problemas de DNS e de carregamento de capas. Os provedores foram isolados por namespace; capas MangaDex passaram a ser entregues pelo backend, com validação e cache limitado. A adição do AniList ampliou a descoberta, mantendo explícita a diferença entre metadados e páginas reais.

Pendências da entrega. Falta anexar a stack Portainer saudável com limite efetivo de 1 GB por container, identificar o commit implantado e repetir integralmente os cenários autenticados no subdomínio. Não foi localizada a issue final "Entrega — Seu negócio no ar". Os logs locais são apresentados também na ficha 11, com autores e horários. Essas pendências mantêm a atividade como parcial.

Outras capturas de leitura. As imagens abaixo complementam a demonstração do leitor com uma ilustração de capa de Tensei Shitara Slime Datta Ken e uma página em português de Kumo desu ga, Nani ka?.



Figura 21: Capa de Tensei Shitara Slime Datta Ken exibida na área de leitura. Captura fornecida pelo aluno, da pasta fotos/.



Figura 22: Página em português de Kumo desu ga, Nani ka?, com identificação do capítulo 73-1. Captura fornecida pelo aluno, da pasta fotos/.

5.5 Atividade 11 — SaaS completo com Stripe

Descrição	Assinatura recorrente, limites Free/Pro, Billing Portal, inadimplência e métricas administrativas.
Data planejada	13/11/2026
Data realizada	13/11/2026 22:24:49[1]
Situação	Parcial; entrega final pendente
Evidência	Commit, Checkout e Portal Stripe teste, limite Free, plano confirmado e auditoria local.
Links diretos	commit `0fe7b05` Matriz de evidências da atividade 11

O que foi feito. O plano passou a depender de assinatura vigente, em vez do booleano Premium legado. A API bloqueia a quarta estante no Free e permite sua criação no Pro. O Billing Portal permite gerenciar a cobrança; a interface apresenta inadimplência, vigência e cancelamento. O admin visualiza usuários por plano, assinaturas ativas, MRR e eventos.

```
Repositório: Paulo794/Manga-Cloud
Commit: 0fe7b0527591aacc167a595a76d173f3951d109d
Autor: Paulo Henrique
Data de autoria: 13/11/2026 22:24:49 -0300
Data do commit: 13/11/2026 22:24:49 -0300
Descrição: feat: consolidado reproducao local, Billing Portal Stripe, metricas admin e modularizacao
```

Figura 23: Print enviado pelo aluno: consolidação de limite, Billing Portal, métricas e reprodução local em 13/11/2026 às 22:24:49.

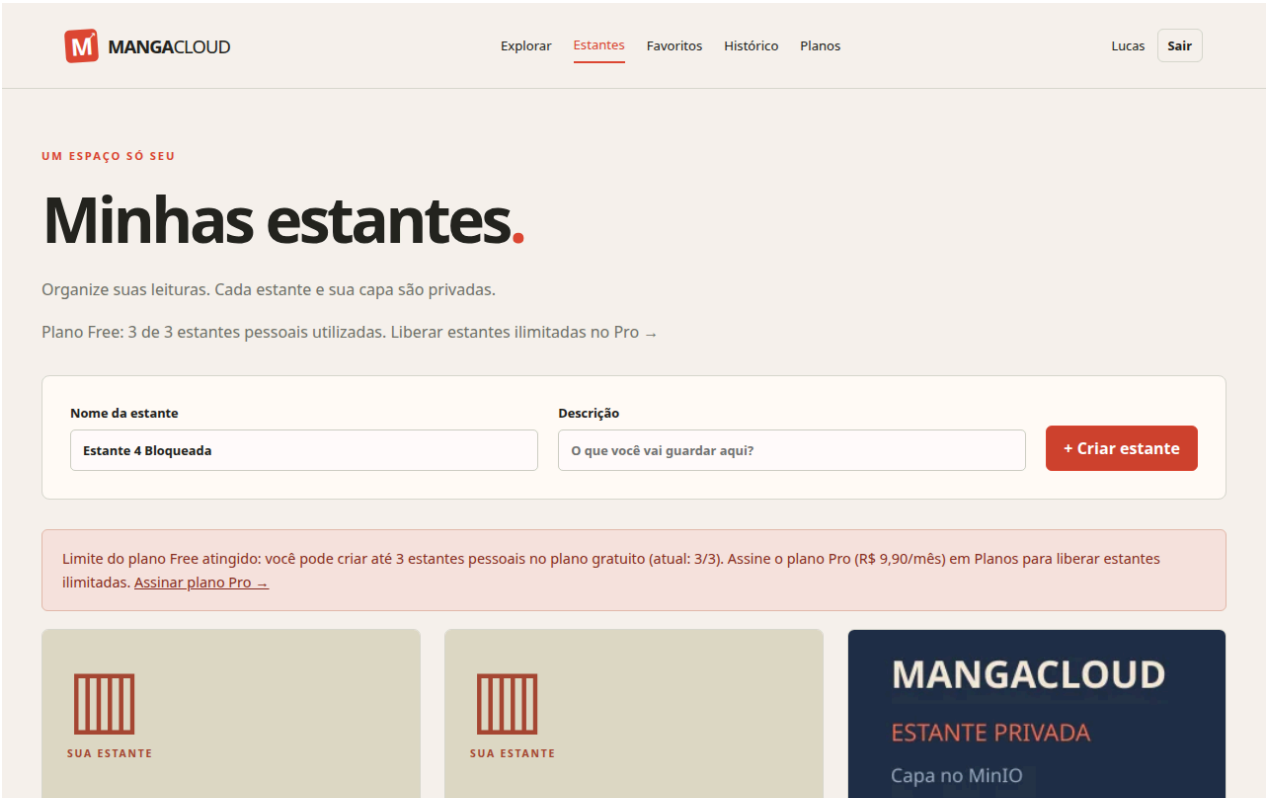


Figura 24: Free com três estantes: a nova criação é recusada com mensagem sobre o limite e o Pro. Captura local, 13/11/2026.

Data e situação. O horário marcado com [1] comprova o commit de implementação. O teste financeiro foi realizado em ambiente local com serviços reais do Stripe em modo de teste; o fluxo público completo e a entrega formal ainda não estão comprovados.

Checkout e confirmação. O cartão 4242 foi utilizado no Checkout hospedado. Após o webhook, a aplicação exibe assinatura Pro ativa. O redirecionamento do navegador não é usado como autorização financeira.

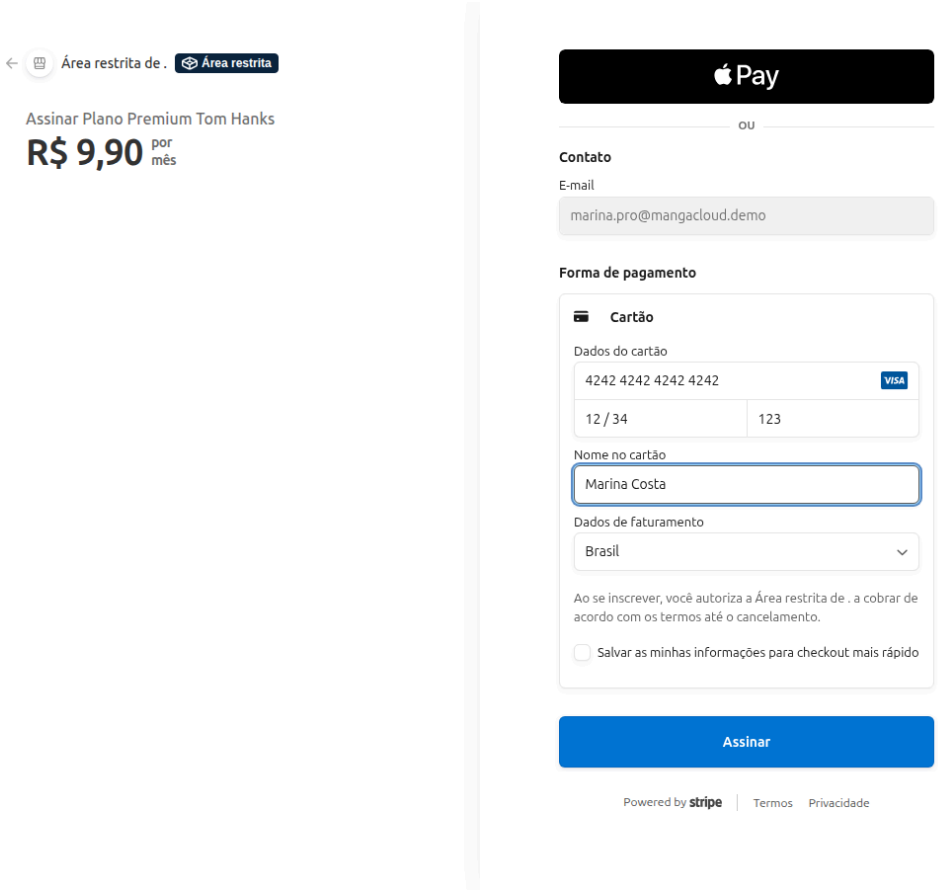


Figura 25: Checkout Stripe em modo teste, R$ 9,90/mês e cartão 4242; nenhum pagamento real.



Figura 26: Plano Pro confirmado no app após webhook, com vigência e botão Gerenciar assinatura. Recorte da captura local.

Portal e ciclo de cancelamento. O Portal real do Stripe foi aberto para a conta de teste. O cancelamento agendado mantém o acesso até o fim da vigência. O retorno ao Free e a renovação recusada foram conferidos nas execuções com Test Clocks registradas no relatório técnico de 13/11.

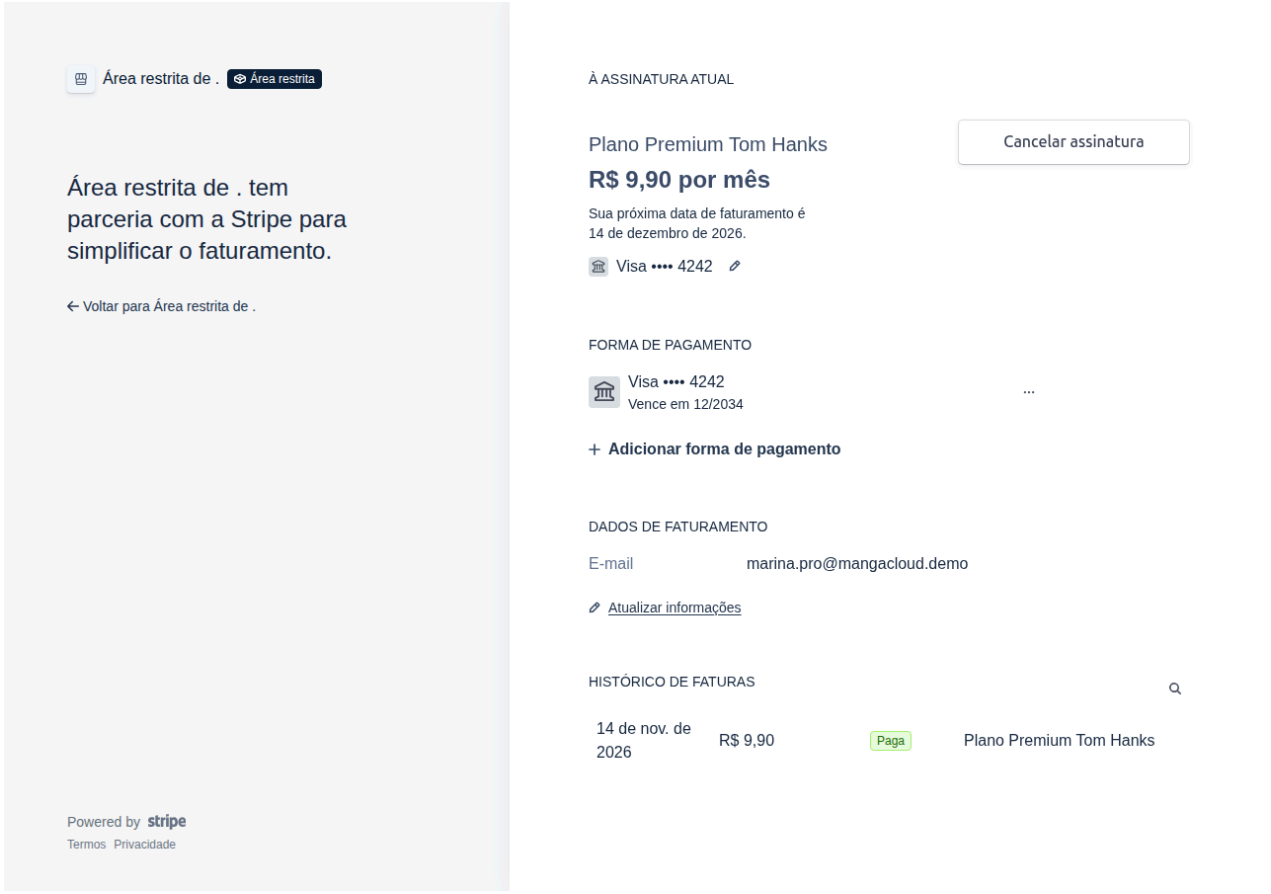


Figura 27: Stripe Billing Portal em modo de teste, com assinatura e forma de pagamento 4242.



Figura 28: App após o cancelamento agendado no Portal, preservando o acesso durante a vigência. Recorte da captura local.

Métricas e eventos. A demonstração local contém quatro contas: três Free e uma Pro, totalizando uma assinatura ativa e MRR de teste de R$ 9,90. Papel e plano aparecem em colunas distintas. Os eventos identificam a liberação por webhook, a abertura do Portal, a atualização da assinatura e acessos negados.

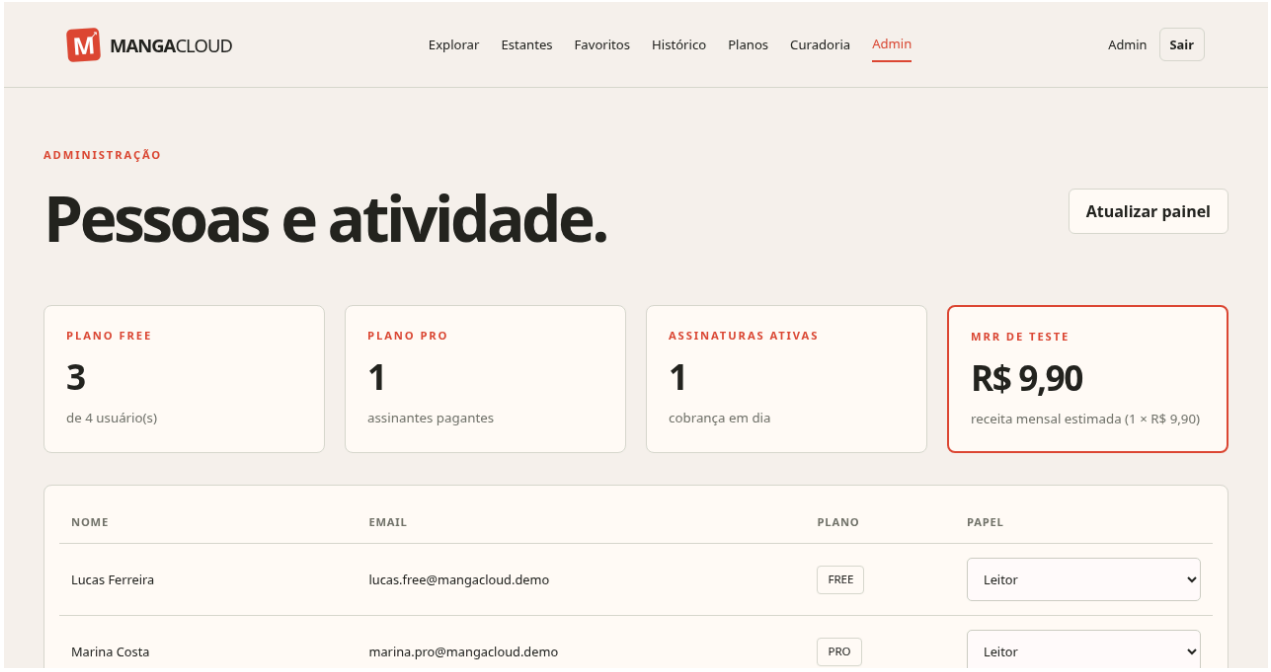


Figura 29: Painel admin local com usuários por plano, assinatura ativa e MRR de teste. Recorte superior.

QUANDO	ATOR	AÇÃO	RECURSO	DETALHE	RESULTADO
13/11/2026, 21:52:30	4	auth.login	—	papel: admin	sucesso
13/11/2026, 21:52:27	3	auth.login	—	papel: curador	sucesso
13/11/2026, 21:52:27	2	assinatura.atualizada	assinatura:sub_1UbVCNEVPVcwLn6e6EUcE5mZ	customer.subscription.updated · PRO (active)	sucesso
13/11/2026, 21:52:26	2	assinatura.renovacao_reativada	assinatura:sub_1UbVCNEVPVcwLn6e6EUcE5mZ	—	sucesso
13/11/2026, 21:52:25	2	assinatura.atualizada	assinatura:sub_1UbVCNEVPVcwLn6e6EUcE5mZ	customer.subscription.updated · PRO (active)	sucesso
13/11/2026, 21:52:24	2	assinatura.atualizada	assinatura:sub_1UbVCNEVPVcwLn6e6EUcE5mZ	customer.subscription.updated · PRO (active)	sucesso
13/11/2026, 21:52:10	2	assinatura.portal_aberto	cliente:cus_VckhA3YieXRoXy	—	sucesso
13/11/2026, 21:52:09	2	assinatura.atualizada	assinatura:sub_1UbVCNEVPVcwLn6e6EUcE5mZ	invoice.payment_succeeded · PRO (active)	sucesso
13/11/2026, 21:52:09	2	assinatura.atualizada	assinatura:sub_1UbVCNEVPVcwLn6e6EUcE5mZ	customer.subscription.created · PRO (active)	sucesso
13/11/2026, 21:52:09	2	assinatura.atualizada	assinatura:sub_1UbVCNEVPVcwLn6e6EUcE5mZ	checkout.session.completed · PRO (active)	sucesso
13/11/2026, 21:52:09	2	assinatura.atualizada	assinatura:sub_1UbVCNEVPVcwLn6e6EUcE5mZ	invoice.paid · PRO (active)	sucesso
13/11/2026, 21:51:55	2	acesso.negado	GET /api/estantes/1	HTTP 404	negado

Figura 30: Auditoria local: horários, atores e eventos Stripe/Portal. Recorte da mesma captura, sem alteração do conteúdo.

Dificuldades e solução técnica. A gestão exigiu tratar eventos repetidos, estado remoto mais recente, cancelamento com `cancel_at` e reativação. A solução combina identidade de cobrança, conferência de propriedade, reconciliação e transação por usuário. Os testes preservam a biblioteca após expiração e verificam que uma falha de renovação não mantém o Pro ativo indevidamente.

Pendências da entrega. Falta uma captura atualizada do Stripe Dashboard web, diferente do Portal do cliente, e a repetição do ciclo completo no subdomínio com webhook público configurado. Não foi localizada a issue final "Entrega — SaaS completo". O relatório não apresenta o Portal como se fosse o Dashboard administrativo do Stripe.

6 Considerações finais

A evolução do projeto demonstra que um SaaS depende da combinação entre regras de negócio e operação. O CRUD de estantes precisou ser protegido por dono; o plano comercial passou a exigir consulta atual ao banco; arquivos receberam validação e acesso privado; eventos financeiros deixaram de ser tratados como um único campo permanente. Essas decisões conectaram as atividades em um produto com responsabilidades mais claras.

O principal aprendizado técnico do período é que identidade, papel e assinatura resolvem problemas diferentes. Uma conta administrativa não deve receber benefícios financeiros por seu papel, e um assinante não deve acessar a biblioteca de outro usuário. A utilização de webhooks, revisões de progresso e transações também tornou explícito o impacto de concorrência, repetição e atraso de mensagens em um sistema distribuído.

O MangaCloud possui implementação funcional e landing acessível publicamente, com evidências locais de catálogo, leitura, estantes, segurança e assinatura de teste. Para completar a entrega operacional, permanecem a inspeção do Portainer, a identificação do commit implantado, as verificações autenticadas completas no domínio e os registros formais ainda ausentes. Para avançar a um produto de uso contínuo, também serão necessários acompanhamento de disponibilidade, rotinas de backup/restauração, testes de carga e uma definição de operação e suporte das fontes externas.

As situações do quadro mantêm visíveis os atrasos e os resultados parciais. A finalidade do relatório é apresentar uma trajetória verificável: o que foi desenvolvido, como os componentes se relacionam, quais provas estão disponíveis e quais pontos ainda precisam ser concluídos.

Declaração de autoria

Declaro que este relatório se refere ao meu projeto e à minha participação individual na disciplina ISW055. As evidências apresentadas correspondem aos arquivos, registros e resultados identificados nos links e nas legendas, com distinção entre histórico, demonstração local e ambiente público. Assumo a responsabilidade pela conferência das informações, pela indicação das limitações e pela revisão do texto antes da entrega.

Paulo Henrique Domingues Antunes

Pompeia, 13/11/2026